

Venus - Deviation Bounded Oracle

Executive Summary

This audit report was prepared by Quantstamp, the leader in blockchain security.

Type	Oracle	Documentation quality	High	
Timeline	2026-04-20 through 2026-04-27	Test quality	High	
Language	Solidity	Total Findings	5	Fixed: 1 Acknowledged: 4
Methods	Architecture Review, Unit Testing, Functional Testing, Computer-Aided Verification, Manual Review	High severity findings ⓘ	1	Fixed: 1
Specification	None	Medium severity findings ⓘ	0	
Source Code	- VenusProtocol/oracle #7ad77af - VenusProtocol/venus-protocol #95b8c6a	Low severity findings ⓘ	2	Acknowledged: 2
Auditors	- Andrei Stefan Auditing Engineer - Alejandro Padilla Auditing Engineer - Cameron Biniamow Auditing Engineer	Undetermined severity findings ⓘ	0	
		Informational findings ⓘ	2	Acknowledged: 2

Summary of Findings

Overall the code is well-written, well-documented, and follows best practices. We have found one significant security vulnerability **VEN-1** that we will recommend to fix together with a few other significant low and informational level.

Fix Review Update: The update indicates that only **VEN-1** and **S1** were explicitly **fixed**, while the rest of the issues are treated as operational considerations or accepted risks.

ID	DESCRIPTION	SEVERITY	STATUS
VEN-1	Transient-Cache Pre-Warm Enables in-Tx Spot Manipulation Bypass on AMM-Pivot Markets	• High ⓘ	Fixed
VEN-2	<code>deviationBoundedOracle</code> Uninitialized During Upgrade, Bricking Core Protocol Functions Until Governance Acts	• Low ⓘ	Acknowledged
VEN-3	Permissionless XVS-to-vXVS Conversion Enables Griefing During Protection	• Low ⓘ	Acknowledged
VEN-4	Lack of Zero-Price Check Could Brick a Market	• Informational ⓘ	Acknowledged
VEN-5	Missing Sanity Bounds on Cooldown and Reset Threshold Can Freeze Markets or Weaken Protection	• Informational ⓘ	Acknowledged

Assessment Breakdown

Disclaimer

Only features that are contained within the repositories at the commit hashes specified on the front page of the report are within the scope of the audit and fix review. All features added in future revisions of the code are excluded from consideration in this report.

Possible issues we looked for included (but are not limited to):

- Transaction-ordering dependence
- Timestamp dependence
- Mishandled exceptions and call stack limits
- Unsafe external calls
- Integer overflow / underflow
- Number rounding errors
- Reentrancy and cross-function vulnerabilities
- Denial of service / logical oversights
- Access control
- Centralization of power
- Business logic contradicting the specification
- Code clones, functionality duplication
- Gas usage
- Arbitrary token minting

Methodology

1. Code review that includes the following
 1. Review of the specifications, sources, and instructions provided to Quantstamp to make sure we understand the size, scope, and functionality of the smart contract.
 2. Manual review of code, which is the process of reading source code line-by-line in an attempt to identify potential vulnerabilities.
 3. Comparison to specification, which is the process of checking whether the code does what the specifications, sources, and instructions provided to Quantstamp describe.
2. Testing and automated analysis that includes the following:
 1. Test coverage analysis, which is the process of determining whether the test cases are actually covering the code and how much code is exercised when we run those test cases.
 2. Symbolic execution, which is analyzing a program to determine what inputs cause each part of a program to execute.
3. Best practices review, which is a review of the smart contracts to improve efficiency, effectiveness, clarity, maintainability, security, and control based on the established industry and academic practices, recommendations, and research.
4. Specific, itemized, and actionable recommendations to help you take steps to secure your smart contracts.

Scope

Files Included

- Pull Request: <https://github.com/VenusProtocol/oracle/pull/306>
- Last commit: [7ad77af](#)
- Files:
 - `contracts/DeviationBoundedOracle.sol`
 - `contracts/interfaces/IDeviationBoundedOracle.sol`
- Pull Request: <https://github.com/VenusProtocol/venus-protocol/pull/668>
- Last commit: [95b8c6a](#)
- Files:
 - `contracts/Oracle/interfaces/IDeviationBoundedOracle.sol` ← new minimal local interface stub (temporary until the oracle package is published with the full interface)
 - `contracts/Comptroller/ComptrollerStorage.sol` (new `ComptrollerV19Storage` contract adding `deviationBoundedOracle` slot)
 - `contracts/Comptroller/Diamond/Diamond.sol`
 - `contracts/Comptroller/ComptrollerInterface.sol` (added `deviationBoundedOracle()` view to the public interface)
 - `contracts/Comptroller/Diamond/facets/FacetBase.sol` (storage upgrade, `_updateProtectionStates` helper, split into mutable vs. view liquidity variants)
 - `contracts/Comptroller/Diamond/facets/PolicyFacet.sol` (bounded price check in `borrowAllowed` ; liquidation and public view paths route to the view-only liquidity variant)
 - `contracts/Comptroller/Diamond/facets/SetterFacet.sol` (new `setDeviationBoundedOracle` governance setter + `NewDeviationBoundedOracle` event)
 - `contracts/Comptroller/Diamond/interfaces/ISetterFacet.sol` (interface update for the new setter)
 - `contracts/Comptroller/Diamond/facets/RewardFacet.sol`
 - `contracts/Tokens/VAI/VAIController.sol` (VAI mint-capacity calculation uses bounded prices)
 - `contracts/Lens/ComptrollerLens.sol` (dual-price routing in `_calculateAccountPosition`)

- `contracts/Lens/SnapshotLens.sol` (account snapshot now returns `spotPrice`, `boundedCollateralPrice`, and `boundedDebtPrice` instead of a single `assetPrice`)
- `contracts/Lens/VenusLens.sol` (`VTokenUnderlyingPrice` struct now exposes `spotPrice`, `collateralPrice`, and `debtPrice`)

Operational Considerations

DeviationBoundedOracle

1. **Window Auto-Expansion is Permissionless and One-Way:** Every successful `updateProtectionState` (or read that warms the cache) widens the window in the direction of the new extreme: spot below current `minPrice` lowers `minPrice`; spot above current `maxPrice` raises `maxPrice`. Windows can only be NARROWED by the keeper (`updateMinPrice` / `updateMaxPrice`) — never by user reads. The trigger anchoring is asymmetric (pump checks against `minPrice`, crash checks against `maxPrice`), so a user-driven slow drift on one side does not progressively weaken defense on that same side.
2. **Transient Cache Scope and Lifetime:** The cache is tx-scoped (uses EVM transient storage / `tstore` slots). The first `updateProtectionState(vToken)` (or first mutable liquidity calc that touches the asset) populates (`cachedMin`, `cachedMax`). All subsequent same-tx calls — both view and mutating — short-circuit on `cachedMin != 0 && cachedMax != 0` BEFORE any fresh spot fetch, window expansion, or trigger evaluation. The cache is not invalidated mid-tx.
3. **Protection Trigger Re-Arms on Every Read:** `_checkAndTriggerProtection` writes `lastProtectionTriggeredAt = block.timestamp` UNCONDITIONALLY whenever the deviation threshold is exceeded — not only on the `false → true` transition. This is intentional (so a sustained attacker can't trivially wait out the cooldown) but means a permissionless caller can refresh the cooldown indefinitely while spot remains outside the window.
4. **Active-Protection Setter Gating is Inconsistent:** When `currentlyUsingProtectedPrice == true`, `setAssetBoundedPricingEnabled(asset, false)` reverts with `ProtectedPriceActive`. However, `setThresholds`, `setCooldownPeriod`, and `exitProtectionMode` carry no equivalent guard — governance can change thresholds, change cooldowns, and call `exitProtectionMode` while protection is active, and can also chain `exitProtectionMode → setAssetBoundedPricingEnabled(false) → setAssetBoundedPricingEnabled(true)` to circumvent the active-protection lock. The active-protection guard is therefore an operational constraint, not a hard invariant. There is no emergency-disable path, so any state where exit is unreachable (trap scenarios, misconfigured cooldown) requires a DBO proxy upgrade. Venus should consider gating the other setters consistently with `setAssetBoundedPricingEnabled` while protection is active.
5. **No Migration Helper for DBO Replacement:** `setDeviationBoundedOracle(newDBO)` on the Comptroller swaps the reference atomically with no on-chain check that newDBO has all currently-bounded markets initialized. No state copy is performed; existing per-asset windows, protection flags, and triggered timestamps are orphaned on the old DBO. Operational discipline (timelock-bundling per-asset init with the swap) is the only defense.
6. **Permissionless Pokes Persist Across Transactions:** `updateProtectionState`, `redeemAllowed`, `transferAllowed`, `claimVenus`, and `claimVenusAsCollateral` are all reachable as DBO-state-mutating entry points without `msg.sender == vToken` or `msg.sender == holder` authorization. Successful calls leave `currentlyUsingProtectedPrice`, `lastProtectionTriggeredAt`, and the window bounds permanently mutated.
7. **DBO Trusts ResilientOracle Verbatim & Pause Cascades to CF Path:** `_fetchSpotPrice` is a single `RESILIENT_ORACLE.getPrice(asset)` call with no zero-check, no staleness filter, no TWAP, no secondary cross-reference. Any malformed (zero) or manipulated value is captured directly into window expansion and trigger evaluation. Operationally, a `ResilientOracle` pause causes every bounded-oracle read to revert — freezing the CF path and breaking lens / analytics queries on every bounded asset. Operators should treat a `ResilientOracle` pause as an effective CF-path-wide pause for bounded assets and document the cascade in runbooks; per-asset pause controls or a graceful stale-price fallback with explicit staleness reporting are alternatives if this blast radius is too broad.
8. **Keeper Liveness is a Hard Operational Dependency:** On-chain logic only widens the window; only the keeper narrows it and clears protection. If the keeper is unavailable during protection, the affected market remains frozen on the CF path until the keeper returns and convergence is achieved. Recommended posture: run redundant keepers across independent infrastructure, monitor missed updates and long-lived protection states, publish the expected cadence, and maintain an operator escalation runbook. Venus should also consider whether a timelock-gated emergency recovery path is appropriate, since `setAssetBoundedPricingEnabled(false)` is blocked during active protection. Convergence during volatile periods is also constrained by the on-chain rule that keeper writes must straddle current spot, so the recovery cadence is a function of both keeper availability and live market movement.

Comptroller Integration

1. **Mutable vs View Liquidity Variants:** `getHypotheticalAccountLiquidityInternal` is now state-changing (calls `_updateProtectionStates`); `getHypotheticalAccountLiquidityInternalView` does not warm the cache. Borrow / redeem / transfer paths use the mutable variant; `liquidateBorrowAllowed` and all public view helpers (`getBorrowingPower`, `getAccountLiquidity`, `getHypotheticalAccountLiquidity`) use the view variant. `redeemAllowedInternal` was a view-callable path pre-PR and is now state-changing — but `redeemAllowed` was already non-view in the Compound-v2 lineage due to XVS flywheel writes, so there is no ABI break.
2. **Dual-Price Routing in `_calculateAccountPosition`:** The CF path (`USE_COLLATERAL_FACTOR`) reads `deviationBoundedOracle` from the Comptroller and uses (`collateralPrice`, `debtPrice`) per market. The LT path (`USE_LIQUIDATION_THRESHOLD`) reads the spot oracle and uses spot for both sides. VAIController's `getMintableVAI` mirrors the CF path semantics for non-VAI assets but adds VAI repay amount as 1:1 USD with no oracle.
3. **Permissionless Protection Activation via the CF Path:** Any function that reaches `_updateProtectionStates(account)` with an arbitrary `account` argument can persistently activate protection on that account's entered markets. `redeemAllowed(vToken, redeemer, 0)` and `transferAllowed(vToken, src, dst, 0)` are externally callable with no `msg.sender == vToken` check, and they return error codes rather than reverting on shortfall — so DBO state mutations persist regardless of liquidity outcome.

Key Actors And Their Capabilities

DeviationBoundedOracle (oracle repo)

- 1. **Users (Permissionless Readers/Pokers):** Any address can call `updateProtectionState(vToken)` to fetch the latest spot, expand the per-asset window in the direction of new extremes, evaluate trigger conditions, and write the resulting `(boundedCollateralPrice, boundedDebtPrice)` pair into the per-tx transient cache. Any address can also call `getBoundedPricesView(vToken)` (and the granular `getBoundedCollateralPriceView / getBoundedDebtPriceView`) to read current bounded prices without state mutation. There is no per-caller authorization on the read or poke surface.
- 2. **Keeper (Off-Chain Bot, ACM-Gated):** Maintains the off-chain 15-minute sliding window for each enabled asset and pushes corrected `minPrice / maxPrice` values via `updateMinPrice(asset, newMin)` and `updateMaxPrice(asset, newMax)`. A 5% deadband (`KEEPER_DEADBAND`) is documented as a write-suppression heuristic but is enforced only by the off-chain helper (`checkAndGetWindowDrift / _exceedsCorrectionDeadband`), not by the on-chain mutating writes. The on-chain writes only require `newPrice != 0, newMin < state.maxPrice, newMax > state.minPrice, newMin ≤ currentSpot, and newMax ≥ currentSpot`. The keeper is a load-bearing actor in the design: on-chain logic widens the window when new extremes are observed, but only the keeper can narrow it and allow protection to clear. If the keeper is unavailable during protection, affected markets remain frozen on the CF path until the keeper returns and the window converges — an availability failure, not a solvency failure, since LT-path liquidations and borrower self-rescue actions (repay, supply) remain functional.
- 3. **Monitor (Off-Chain Bot, ACM-Gated):** Calls `exitProtectionMode(asset)` to clear `currentlyUsingProtectedPrice` once two conditions hold simultaneously: (a) `block.timestamp ≥ lastProtectionTriggeredAt + cooldownPeriod`, and (b) `(maxPrice - minPrice) × 1e18 / minPrice < resetThreshold`. Both gates are strict; the monitor cannot exit until both pass.
- 4. **DBO Governance (ACM-Gated, via Venus Normal Timelock):** Controls per-asset onboarding and configuration. Can call `setTokenConfig(asset, ...)` to seed an asset's window from the current spot, `setAssetBoundedPricingEnabled(asset, bool)` to toggle bounded pricing (only when protection is inactive on disable), `setThresholds(asset, trigger, reset)` to update the trigger/reset thresholds, and `setCooldownPeriod(asset, newCooldown)` to change the per-asset cooldown. There is no on-chain timelock on these setters beyond the upstream Venus governance flow. Bounded-pricing configuration is safety-critical and should be reviewed with the same care as collateral-factor changes; threshold and cooldown updates are also reachable while protection is active (see Operational Considerations).
- 5. **ResilientOracle (Upstream Dependency):** The sole spot-price source consumed by `_fetchSpotPrice`. DBO trusts its return verbatim — no zero-check, no staleness filter, no cross-reference. If `ResilientOracle` is paused or unavailable, every `getBoundedPricesView / updateProtectionState` call reverts in turn, freezing the CF path on every bounded asset and breaking lens reads that depend on bounded prices. Under the current implementation, zero prices from `ResilientOracle` do not propagate into DBO because every observed `getPrice` return path rejects zero — a reliance that is not enforced inside DBO itself.

Comptroller Integration (venus-protocol repo)

- 1. **Borrowers / Suppliers / Redeemers (Mutable CF Path):** Any user calling `vToken.borrow, vToken.redeem, vToken.repayBorrow, vToken.transfer, or vToken.mint` indirectly invokes the mutable `getHypotheticalAccountLiquidityInternal`, which calls `FacetBase._updateProtectionStates(account)`. That helper iterates `accountAssets[account]` and calls `deviationBoundedOracle.updateProtectionState(vToken_i)` for each entered market — warming the transient price cache for the whole tx and possibly activating protection on any market with deviating spot. Repay, supply, and seize flows do not run the bounded-pricing CF check, preserving borrower self-rescue during active protection. The user-visible impact of an active protection event is therefore a temporary loss of CF-path availability (borrow / redeem / transfer / VAI mint) on the affected market; pure suppliers are less affected, although in practice many UIs auto-enter markets.
- 2. **Liquidators (LT Path):** `liquidateBorrowAllowed` routes to `getHypotheticalAccountLiquidityInternalView` and uses spot prices (`oracle.getUnderlyingPrice(vToken)`) for both collateral and debt. The LT path is unchanged by this PR — liquidations always evaluate at spot.
- 3. **VAI Minters (Core Pool Only):** `VAIController.mintVAI` is gated to `userPoolId == CORE_POOL_ID`. `getMintableVAI` now calls `boundedOracle.getBoundedPricesView(asset)` per entered market and uses `collateralPrice` for collateral weighting and `debtPrice` for the existing-borrow weighting.
- 4. **External Callers (Permissionless Reward Claims):** `RewardFacet.claimVenusAsCollateral(holder)` and `RewardFacet.claimVenus(..., collateral = true)` are permissionless and route through `grantXVSInternal`. If the targeted holder shows shortfall under the CF-path liquidity check (which uses bounded prices during protection), accrued XVS is force-converted into vXVS via `mintBehalf`. Pre-PR, the bankrupt branch only applied to accounts genuinely insolvent at spot prices; with bounded pricing the eligible set expands to include accounts that are not actually liquidatable, creating a new grieving surface.
- 5. **Comptroller Governance (ACM-Gated):** `SetterFacet.setDeviationBoundedOracle(newDBO)` swaps the Comptroller's `deviationBoundedOracle` storage variable atomically. Only checks are ACM gating, no-op rejection, and `ensureNonzeroAddress`. There is no per-market verification, no migration helper, and no on-chain coupling between market listing in the Comptroller and DBO initialization. Operationally, the storage upgrade exposing `deviationBoundedOracle` and the corresponding `setDeviationBoundedOracle` call must be sequenced atomically — leaving the slot at `address(0)` between proposals breaks every CF-path action and lens read until the setter is invoked.
- 6. **Lens Consumers (Off-Chain):** `SnapshotLens.getAccountSnapshot` exposes `spotPrice, boundedCollateralPrice, and boundedDebtPrice`. `VenusLens.vTokenUnderlyingPrice` was specified to expose the same triple but the on-disk implementation still returns a single `underlyingPrice` field — a scope/code mismatch.

Findings

VEN-1

Transient-Cache Pre-Warm Enables in-Tx Spot Manipulation Bypass on AMM-Pivot Markets

• High ⓘ Fixed

i Update

The client stated that the specific flag that was added is initialised as false and will only be flipped after a risk analysis is done per asset.

File(s) affected: `contracts/DeviationBoundedOracle.sol`, `contracts/Comptroller/Diamond/facets/PolicyFacet.sol`, `contracts/Comptroller/Diamond/facets/FacetBase.sol`

Description: `DeviationBoundedOracle._updateAndGetBoundedPrices` at L#604-605 and `_computeBoundedPrices` at L#701-702 short-circuit on `(cachedMin != 0 && cachedMax != 0)` BEFORE any fresh `_fetchSpotPrice`, window expansion, or trigger evaluation. When protection is INACTIVE (the steady-state default), `_resolveBoundedPrices` returns `(spot, spot)` with no clamping on either side, and `_setCachedPrices` writes both slots. On the very next same-tx read, the cache returns the same scalar even if the live spot has moved arbitrarily within that tx.

This defeats the first-principles guarantee that bounded pricing can never be weaker than spot-only pricing. An attacker with collateral on an asset whose `ResilientOracle` active feed is in-tx manipulable (AMM pivot or short-TWAP pivot) can:

1. Pre-warm the cache at the natural high spot via the permissionless `dbo.updateProtectionState(vX)` (no clamping; `(spot, spot)` cached).
2. Manipulate the underlying AMM in the same tx (flash-loan funded dump pushing spot to a depressed value).
3. Borrow against the cached high collateral price — the borrow path's zero-guard reads `getBoundedPricesView` (cache HIT, returns the stale high spot) and the mutable warm-up inside `_updateProtectionStates` ALSO short-circuits on the cache, so no fresh fetch / window expansion / trigger is performed against the manipulated spot.
4. Default. Liquidation later seizes at the post-dump spot (LT path uses spot directly), under-recovering the debt.

Exploit Scenario:

1. State: `isBoundedPricingEnabled = true`, `currentlyUsingProtectedPrice = false`, window `(95, 110)`, spot `S1 = $100`. Attacker has 1,000 `vX` supplied.
2. Attacker calls `dbo.updateProtectionState(vX)` — DBO fetches spot (\$100), window unchanged, no trigger, cache writes `(100, 100)`.
3. Same tx: attacker flash-loans and dumps the AMM pool serving as `ResilientOracle`'s pivot, pushing the live source to `S2 = $50`.
4. Same tx: attacker calls `vX.borrow(USDC, 65_000e18)`. Inside `borrowAllowed`:
 - Zero-price guard `dbo.getBoundedPricesView(vX)` → cache HIT → returns `(100, 100)`. Passes.
 - Liquidity check `getHypotheticalAccountLiquidityInternal(...)` → `_updateProtectionStates(attacker)` → `dbo.updateProtectionState(vX)` → cache HIT → immediate return. NO fresh spot fetched, NO window expansion against the \$50 spot, NO trigger evaluation.
 - `ComptrollerLens._calculateAccountPosition(CF)` → `dbo.getBoundedPricesView(vX)` → cache HIT → `(100, 100)`. `sumCollateral = 100 × 1,000 × CF = $70,000`. `sumBorrow = $65,000`. Approved.
5. USDC transferred. Attacker repays the flash loan and retains the net.
6. Subsequent block: liquidator sees the position underwater at spot \$50. LT path seizes `vX` at spot. Recovery $\leq \$50 \times 1,000 \times (1 - \text{incentive}) \approx \$45,000$. Bad debt $\approx \$20,000$ per attacker position.

Recommendation: Gate the cache short-circuit on `currentlyUsingProtectedPrice == true` only. In the inactive-protection path, always re-fetch spot, run `_expandPriceWindow`, and re-evaluate `_checkAndTriggerProtection`.

VEN-2

`deviationBoundedOracle` **Uninitialized During Upgrade, Bricking Core Protocol Functions Until Governance Acts** • Low ⓘ Acknowledged

i Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

The VIP applying the diamond cut will bundle `setDeviationBoundedOracle` atomically (in the same transaction), so the slot is never observable as `address(0)`.

File(s) affected: `contracts/Comptroller/ComptrollerStorage.sol`, `contracts/Comptroller/Diamond/facets/FacetBase.sol`, `contracts/Comptroller/Diamond/facets/PolicyFacet.sol`, `contracts/Comptroller/Diamond/facets/MarketFacet.sol`, `ontracts/Lens/ComptrollerLens.sol`, `contracts/Tokens/VAI/VAIController.sol`

Description: `ComptrollerV19Storage` introduces a new storage field for the `DeviationBoundedOracle`. This field defaults to `address(0)` when the Diamond upgrade is applied. The only mechanism to populate it is a subsequent governance call to `SetterFacet.setDeviationBoundedOracle`. There is no initializer, no constructor argument, and no mechanism to atomically set this value as part of the Diamond cut itself.

Therefore, while unset, every code path that calls into the DBO performs an unchecked external call to `address(0)`. The affected entry points and their call chains are:

- `borrowAllowed` (PolicyFacet.sol:133): Every new borrow attempt across every market fails.
- `redeemAllowed` → `redeemAllowedInternal` → `getHypotheticalAccountLiquidityInternal` → `_updateProtectionStates` (FacetBase.sol:308-313): Every redemption by a user who has entered at least one market fails.
- `enterPool` → `getHypotheticalAccountLiquidityInternal` → `_updateProtectionStates` (MarketFacet.sol:362, FacetBase.sol:164): Every pool selection or switch fails.
- `ComptrollerLens._calculateAccountPosition` (ComptrollerLens.sol:240): Any liquidity check on the CF path fails, including those invoked transitively from `borrowAllowed`, `redeemAllowed`, and `enterPool`.
- `VAIController.getMintableVAI` (VAIController.sol:464): All VAI minting fails.

The combined effect is a complete denial-of-service for the four most critical user-facing operations in the protocol. It impacts every user across every market simultaneously from the moment the upgrade is applied. The protocol would be non-functional for the duration of the governance delay.

Recommendation: The simplest mitigation is to include the `setDeviationBoundedOracle` call in the same transaction as the Diamond upgrade, either by batching it in a multisig or by deploying a dedicated upgrade script that performs both operations atomically. This eliminates the window entirely.

If atomic execution is not achievable given the governance setup, consider keeping the existing spot oracle as a fallback on the CF path when `deviationBoundedOracle` is unset, matching pre-upgrade behavior and ensuring no regression in liveness during the configuration window.

VEN-3 Permissionless XVS-to-vXVS Conversion Enables Griefing During Protection

• Low ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

The attack tops out at griefing, no funds can be stolen. The accounts targetable by this attack are those that are solvent at spot prices but appear insolvent under bounded pricing. Liquidators cannot touch them, because liquidation uses spot prices (LT path), not bounded prices. The force-minted vXVS is also not seizable: `mintBehalf` does not enter the holder into the vXVS market, so `seizeAllowed` rejects with `MARKET_NOT_COLLATERAL`. The holder keeps the full value of their XVS through vXVS and can redeem at any time.

Description: Permissionless reward-claim functions in `RewardFacet.sol` allow any caller to claim rewards on behalf of any holder:

```
- `claimVenusAsCollateral(holder)`  
- `claimVenus(..., collateral = true)`
```

Both functions route through `grantXVSInternal`. If the targeted holder has a positive shortfall under the collateral-factor liquidity check, `grantXVSInternal` treats the holder as bankrupt and mints vXVS to them instead of transferring accrued XVS directly.

During an active protection event, collateral-factor liquidity is computed using bounded prices: collateral is valued at the lower bound and debt at the upper bound. This means an account that remains solvent at spot prices can still appear insolvent under bounded pricing.

As a result, any third party can force-convert a bounded-insolvent holder's accrued XVS into vXVS without the holder's consent.

The impact is limited to griefing. The holder keeps the economic value through their vXVS balance and can redeem it back to XVS. `mintBehalf` also does not automatically enter the holder into the vXVS market, so redemption is not blocked by the bounded-pricing shortfall check for holders who are not market members.

However, the holder may still incur friction, including redeem gas costs, exchange-rate drift between mint and redeem, possible tax implications, and a temporary voting-power gap until redemption.

This permissionless claim-as-collateral path predates the bounded-oracle integration. Previously, the bankrupt branch only applied to accounts genuinely insolvent at spot prices. With bounded pricing, the eligible set expands to include accounts that are not actually liquidatable, creating a new griefing surface.

Recommendation: Decide whether this behavior is an acceptable side effect of bounded pricing. If not, restrict the permissionless claim-as-collateral path. For example, require holder consent, limit third-party calls to accounts with spot-price shortfall, or otherwise ensure bounded-pricing-only shortfall cannot trigger forced XVS-to-vXVS conversion.

VEN-4 Lack of Zero-Price Check Could Brick a Market

• Informational ⓘ Acknowledged

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

DBO is a protection layer on top of ResilientOracle — it does not produce prices, only constrains them. Zero-price rejection is a ResilientOracle invariant, so re-asserting it inside the protection layer would duplicate the upstream guarantee.

File(s) affected: `contracts/DeviationBoundedOracle.sol`

Description: `DeviationBoundedOracle` does not explicitly reject zero prices before writing to `minPrice` or `maxPrice`.

The internal `_safeToUint128` helper only protects against overflow. It still treats `0` as a valid value. As a result, both the initial seeding path in `_setTokenConfig` and the new-low update path in `_expandPriceWindow` can write a zero price if the upstream oracle ever returns one.

If a zero price is written, the affected market can become effectively bricked:

- The protection trigger bound, ``minPrice * (1 + triggerThreshold)``, collapses to zero.
- Any positive spot price will therefore trigger price protection.
- ``_computePriceBoundRatio`` will revert due to division by zero.
- The bounded collateral price becomes ``min(spot, 0) = 0``, causing all collateral for the asset to stop counting.
- ``setAssetBoundedPricingEnabled(false)`` cannot be used to recover, because the ``ProtectedPriceActive`` guard blocks disabling bounded pricing while protection is active.

This leaves a contract upgrade as the only practical recovery path.

This issue is **not currently exploitable**. The current upstream oracle, `ResilientOracle.getPrice()`, rejects zero prices on every observed return path, so the price source consumed by `DeviationBoundedOracle` should not return zero today.

However, this is a defense-in-depth concern. `DeviationBoundedOracle` does not enforce the zero-price invariant itself. If `ResilientOracle` changes in the future, or if `DeviationBoundedOracle` is later pointed to a different upstream oracle that returns zero instead of reverting, a zero price could be silently written into `state.minPrice` and brick the affected market.

Recommendation: Add an explicit zero-price check before writing to `minPrice` or `maxPrice`. For example, reject zero prices in both the configuration seeding path and the price-window expansion path before calling `_safeToUint128`

VEN-5

Missing Sanity Bounds on Cooldown and Reset Threshold

Can Freeze Markets or Weaken Protection

Informational ⓘ

Acknowledged

i

Update

Marked as "Acknowledged" by the client.
The client provided the following explanation:

Governance-controlled via the normal timelock and editable post-set, so misconfiguration is recoverable

Description: Two per-asset `DeviationBoundedOracle` configuration parameters lack meaningful sanity bounds.

- `cooldownPeriod` is only required to be greater than zero. There is no practical upper bound, and no lower bound beyond rejecting zero. As a result, an admin typo or unit mistake could set the cooldown extremely low, removing the intended minimum protection period, or extremely high, leaving the affected market frozen on the collateral-factor path for an unreasonable amount of time.
- `resetThreshold` is only required to be greater than zero and lower than `triggerThreshold`. This allows values that are technically valid but operationally unsafe. For example, a mantissa mistake such as setting `50` instead of `50e16` would make the reset condition almost impossible to satisfy, potentially keeping protection active indefinitely. Conversely, a value just below `triggerThreshold` could allow protection to exit as soon as the cooldown expires, even if prices have not meaningfully stabilized.

These values are governance-controlled, so the issue requires admin misconfiguration. Recovery is also straightforward through a setter call. No funds are directly at risk; the impact is either reduced availability, through frozen CF-path markets, or weakened protection due to premature exits.

`triggerThreshold` is not affected, as it is already bounded by the `MIN_THRESHOLD` and `MAX_THRESHOLD` constants.

Recommendation: Add explicit minimum and maximum bounds for `cooldownPeriod` and `resetThreshold` in `setCooldownPeriod`, `setThresholds`, and `_setTokenConfig`. The exact production values are a design decision, but they should reject settings outside operationally reasonable ranges. Indicative bounds could be:

- `60 seconds <= cooldownPeriod <= 1 week`
- `0.5% <= resetThreshold <= triggerThreshold - 0.5%`

The goal is to prevent configuration mistakes from silently freezing markets or weakening the intended protection guarantees.

Auditor Suggestions

S1

COLLATERAL_PRICE_CACHE_SLOT

Constant Diverges From Its Documented ERC-7201

Fixed

Derivation



Update

Marked as "Fixed" by the client.
Addressed in: d328cd62f9fdb6b278add6686d36a18a368bd5b0 .

Description: The NatSpec above COLLATERAL_PRICE_CACHE_SLOT (DeviationBoundedOracle.sol:54–59) states that the constant is derived using the canonical ERC-7201 formula from the namespace "venus-protocol/oracle/DeviationBoundedOracle/cache" . However, manually calculating the slot yields 0x3f67a9b32f658d814e1bad90f322c4180af8aac953a3b543791b0f6c2788a700 , not the expected 0x818cfa9b1e1b1cc716656acdb79a94121ed79bfb196bf958683ed2a3277cb200 . Currently, this does not have any runtime impact.

Recommendation: Address the discrepancy for consistency and to avoid potential issues in the future.

Definitions

- High severity** – High-severity issues usually put a large number of users' sensitive information at risk, or are reasonably likely to lead to catastrophic impact for client's reputation or serious financial implications for client and users.
- Medium severity** – Medium-severity issues tend to put a subset of users' sensitive information at risk, would be detrimental for the client's reputation if exploited, or are reasonably likely to lead to moderate financial impact.
- Low severity** – The risk is relatively small and could not be exploited on a recurring basis, or is a risk that the client has indicated is low impact in view of the client's business circumstances.
- Informational** – The issue does not pose an immediate risk, but is relevant to security best practices or Defence in Depth.
- Undetermined** – The impact of the issue is uncertain.
- Fixed** – Adjusted program implementation, requirements or constraints to eliminate the risk.
- Mitigated** – Implemented actions to minimize the impact or likelihood of the risk.
- Acknowledged** – The issue remains in the code but is a result of an intentional business or design decision. As such, it is supposed to be addressed outside the programmatic means, such as: 1) comments, documentation, README, FAQ; 2) business processes; 3) analyses showing that the issue shall have no negative consequences in practice (e.g., gas analysis, deployment settings).

Test Suite Results

npx hardhat test

```
AnkrBNBOracle unit tests
  deployment
    ✓ revert if ankrBNB address is 0
    ✓ revert if ResilientOracle address is 0
    ✓ should deploy contract
  getPrice
    ✓ revert if ankrBNB address is wrong
    ✓ should get correct price

BNBxOracle unit tests
  deployment
    ✓ revert if stakeManager address is 0
    ✓ revert if BNBx address is 0
    ✓ revert if resilientOracle address is 0
    ✓ should deploy contract
  getPrice
    ✓ revert if BNBx address is wrong
    ✓ should get correct price

Binance Oracle unit tests
  ✓ set price
```


- ✓ **set** BNB price
- ✓ **fetch** price
- ✓ **fetch** BNB price
- ✓ price expired
- ✓ **set** WBETH price
- ✓ **fetch** WBETH price
- ✓ revert **when** setting feed registry address **and** sid already available
- ✓ revert **when** feed registry address **is** zero
- ✓ **fetch** price **from** direct feed registry

bound **validator**

- add** validation config
 - ✓ length **check**
 - ✓ validation config **check**
 - ✓ config added successfully & event **check**
- validate** price
 - ✓ **validate** price (66ms)

Oracle unit tests

- set** token config
 - ✓ cannot **set** feed **to** zero address
 - ✓ sets a token config
- batch **set** token configs
 - ✓ cannot **set** feed **or** vtoken **to** zero address
 - ✓ parameter length **check**
 - ✓ **set** multiple feeds

getPrice

- ✓ gets the price **from** Chainlink **for** vBNB
- ✓ gets the price **from** Chainlink **for** USDC
- ✓ gets the price **from** Chainlink **for** USDT
- ✓ gets the price **from** Chainlink **for** DAI
- ✓ gets the direct price **of** a **set** asset
- ✓ reverts **if no** price **or** feed has been **set**

setDirectPrice

- ✓ sets the direct price

stale price validation

- ✓ stale price period cannot be 0
- ✓ modify stale price period will emit an event
- ✓ revert **when** price stale
- ✓ **if** updatedAt **is some time in** the future, revert it
- ✓ the chainlink answer **is** 0, revert it

DeviationBoundedOracle

constructor

- ✓ sets immutables correctly
- ✓ reverts **when** _resilientOracle **is** zero address
- ✓ reverts **when** nativeMarketAddress **is** zero address

initialize

- ✓ sets **access** control manager

setTokenConfig

happy **path**

- ✓ sets **all** struct fields, emits events, updates asset lists
- ✓ initializes **with** bounded pricing disabled **when** enableBoundedPricing **is false**

revert branches

- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** asset **is** zero address
- ✓ reverts **when** already initialized
- ✓ reverts **when** threshold < MIN_THRESHOLD
- ✓ reverts **when** threshold > MAX_THRESHOLD
- ✓ reverts **when** resetThreshold >= triggerThreshold
- ✓ reverts **when** asset **is** VAI
- ✓ reverts **when** cooldownPeriod **is** zero
- ✓ reverts **when** triggerThreshold **is** zero
- ✓ reverts **when** resetThreshold **is** zero
- ✓ reverts **when** re-initializing **after** de-whitelist
- ✓ reverts **with** PriceExceedsUint128 **when** oracle **returns** > uint128 max

setTokenConfigs (batch)

- ✓ batch-initializes multiple assets, verifies structs **and** events
- ✓ batch **with** mixed enableBoundedPricing **values**
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** **array** lengths mismatch
- ✓ reverts **when** one asset **in** batch **is** invalid (entire tx reverts)

- ✓ reverts **when** one asset **is** already initialized
- ✓ succeeds **with** empty arrays (**no-op**)

setCooldownPeriod

- ✓ updates cooldownPeriod **and** emits event
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** asset **is** zero address
- ✓ reverts **when not** initialized
- ✓ reverts **when** **new** cooldown **is** zero

setThresholds

- ✓ updates **both** thresholds **and** emits **both** events
- ✓ **only** emits TriggerThresholdSet **when only trigger** changes
- ✓ **only** emits ResetThresholdSet **when only reset** changes
- ✓ emits **no** events **when** neither changes
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** asset **is** zero address
- ✓ reverts **when not** initialized
- ✓ reverts **when trigger** below MIN_THRESHOLD
- ✓ reverts **when trigger** above MAX_THRESHOLD
- ✓ reverts **when reset** >= **trigger** (InvalidResetThreshold)
- ✓ reverts **when** triggerThreshold **is** zero
- ✓ reverts **when** resetThreshold **is** zero

setAssetBoundedPricingEnabled

- ✓ disables bounded pricing **and** emits event
- ✓ enables bounded pricing **and** emits event
- ✓ setAssetBoundedPricingEnabled(**true**) **on** already-enabled **is** a **no-op**
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** asset **is** zero address
- ✓ reverts **when not** initialized
- ✓ reverts **when** disabling **with** active protection
- ✓ price **functions return** spot **after** disabling bounded pricing

updateMinPrice

- ✓ updates minPrice **and** emits event
- ✓ allows setting price **within** keeper deadband (**not** enforced **on-chain**)
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** asset **is** zero address
- ✓ reverts **when** price **is** zero
- ✓ reverts **when not** initialized
- ✓ reverts **when** newMin > currentSpot
- ✓ reverts **when** newMin >= maxPrice

updateMaxPrice

- ✓ updates maxPrice **and** emits event
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** asset **is** zero address
- ✓ reverts **when** price **is** zero
- ✓ reverts **when not** initialized
- ✓ reverts **when** newMax < currentSpot
- ✓ reverts **when** newMax <= minPrice

exitProtectionMode

- ✓ disables protection **after** governance raises **reset** threshold
- ✓ prices revert **to** spot **after** exitProtectionMode (**42ms**)
- ✓ reverts **when** caller **is** unauthorized
- ✓ reverts **when** protection **is not** active
- ✓ reverts **when** cooldown has **not** elapsed
- ✓ reverts **when** range **not** converged
- ✓ reverts **when** rangeRatio exactly at resetThreshold (uses >=)
- ✓ succeeds **when** rangeRatio at resetThreshold - **1** wei

getBoundedCollateralPrice

- ✓ **returns** spot **when not** whitelisted
- ✓ **returns** spot **when** whitelisted, **no** deviation
- ✓ expands **window** **when** spot < minPrice
- ✓ expands **window** **when** spot > maxPrice
- ✓ triggers protection **on** pump **and returns** minPrice (exact arithmetic)
- ✓ triggers protection **on** crash **and returns** spot (exact arithmetic)
- ✓ expands **window** min **and** max **while** protection **is** already active
- ✓ **returns** spot **for** uninitialized asset (setTokenConfig never **called**)
- ✓ reverts **when** vToken **is** zero address
- ✓ resolves native market **to** NATIVE_TOKEN_ADDR
- ✓ reverts **with** PriceExceedsUint128 **when** oracle **returns** > uint128 max

getBoundedDebtPrice

- ✓ **returns** spot **when not** whitelisted
- ✓ **returns** spot **when** no deviation
- ✓ **returns** spot **on** pump **trigger** (spot > max **after** expansion)

- ✓ **returns** maxPrice **on** crash **trigger** (spot < max)

- ✓ **reverts when** vToken **is** zero address

getBoundedPrices

- ✓ **returns** (spot, spot) **when not** whitelisted

- ✓ **returns** (spot, spot) **when** whitelisted, **no** deviation

- ✓ **returns** (minPrice, maxPrice) **when** protection active (pump scenario)

- ✓ **both values** match individual getBoundedCollateralPrice **and** getBoundedDebtPrice

view price functions

- ✓ **returns** spot **when not** whitelisted

- ✓ **returns** spot **when no** protection **and no** deviation

- ✓ **returns** bounded price **when** protection **is** stored active (38ms)

- ✓ simulated **trigger: returns** bounded price **without** state mutation

- ✓ **view and non-view return** identical prices **when** protection **is** active (pump)

- ✓ **view and non-view return** identical prices **when** protection **is** active (crash) (38ms)

getBoundedPricesView

- ✓ **returns** same **as** individual **view functions**

- ✓ works **with cache** (after updateProtectionState) (45ms)

updateProtectionState

- ✓ expands **window and** triggers protection **for** whitelisted asset

- ✓ **is** a **no-op for** non-whitelisted asset

- ✓ **reverts with** PriceExceedsUint128 **when** oracle **returns** > uint128 max

- ✓ expands **window without** triggering protection

- ✓ triggers protection **without** expanding **window**

isBoundedPricingEnabled

- ✓ **returns true for** enabled asset

- ✓ **returns false for** disabled asset

- ✓ **returns false for** uninitialized asset

currentlyUsingProtectedPrice

- ✓ **returns true when** protection **is** active

- ✓ **returns false when** protection **is not** active

- ✓ **returns false for** uninitialized asset

canExitProtection

- ✓ **returns false when** protection **is not** active

- ✓ **returns false for** uninitialized asset

- ✓ **returns false before** cooldown elapsed

- ✓ **returns false when** range **not** converged (cooldown elapsed but range wide)

- ✓ **returns true when** cooldown elapsed **and** range converged

- ✓ **returns false when** rangeRatio exactly at resetThreshold (boundary **precision**)

getInitializedAssets

- ✓ **returns all** initialized assets

- ✓ includes de-whitelisted assets

- ✓ **returns empty when none**

getAllBoundedPricingEnabledAssets

- ✓ **returns** correct filtered **array**

- ✓ **returns empty when none** whitelisted

- ✓ filters correctly **after** partial de-whitelist (79ms)

checkAndGetWindowDrift

- ✓ **returns true when** drift > KEEPER_DEADBAND

- ✓ **returns false when** drift <= KEEPER_DEADBAND

- ✓ **returns false when both** prices equal (drift = 0)

- ✓ **returns false when current or** proposed price **is** zero

- ✓ **returns false when** drift exactly at KEEPER_DEADBAND (**strict** >)

- ✓ **returns true when** drift at KEEPER_DEADBAND + 1 wei

- ✓ covers **both** drift directions: proposed above **and** below **on-chain**

- ✓ **returns [false, false] for** uninitialized asset **with** non-zero proposed **values**

- ✓ **reverts when array** lengths mismatch

assetProtectionConfig getter

- ✓ **returns all** struct fields correctly

volatile price extends protection period

- ✓ continued deviation updates lastProtectionTriggeredAt, extending cooldown (45ms)

- ✓ protection period does **NOT** extend **when** price **returns within** threshold

DeviationBoundedOracle E2E

E2E-1: updateProtectionState → **view price functions** (transient **cache**)

- ✓ 1a: **cache** hit, no protection – view returns cached spot

- ✓ 1b: **cache** hit, protection already active – view returns bounded prices

- ✓ 1c: **cache** hit, protection triggered + window expanded in same call

- ✓ 1d: **cache** miss, no deviation – view fetches fresh, returns spot

- ✓ 1e: **cache** miss, exceeds deviation – simulated trigger returns bounded price

- ✓ 1f: per-asset **cache** isolation – update A, view B has no **cache**

- ✓ 1g: non-whitelisted – updateProtectionState caches (spot, spot), views **return** cached spot

E2E-2: protection disabled → extreme price **move**

- ✓ 2a: non-view re-triggers **after disable** (46ms)
- ✓ 2b: **view** simulates **trigger after disable**
- ✓ 2c: lastProtectionTriggeredAt is fresh **after re-trigger** (39ms)
- ✓ 2d: repeated **trigger** → **disable** → **trigger cycle** (84ms)

E2E-3: threshold **update** effects on prices

- ✓ 3a: lowering threshold triggers previously-safe asset
- ✓ 3b: raising threshold prevents **trigger**
- ✓ 3c: **view** reflects threshold change immediately
- ✓ 3d: **reset** threshold interaction with exitProtectionMode

E2E-4: **no re-trigger when active, window** still expands

- ✓ 4a: continued deviation updates lastProtectionTriggeredAt **and** re-emits event
- ✓ 4b: **window** expands during active protection, deviation re-emits event (38ms)
- ✓ 4c: bounded prices reflect expanded **window**

E2E-5: keeper updates during protection

- ✓ 5a: keeper updates succeed during active protection
- ✓ 5b: keeper **update** succeeds **after disable** (40ms)
- ✓ 5c: keeper min **update** during protection affects bounded prices

E2E-6: collateral vs debt price divergence under protection

- ✓ 6a: spot **between** min **and** max – collateral = MIN_PRICE, debt = maxPrice
- ✓ 6b: spot below min – collateral = spot, debt = maxPrice
- ✓ 6c: spot above max – collateral = MIN_PRICE, debt = spot

E2E-7: multiple assets independence

- ✓ 7a: **trigger on** one, other unaffected (47ms)

E2E-8: setAssetBoundedPricingEnabled round-trip

- ✓ 8a: **disable returns** spot, re-enable resets **window to current** spot

E2E-10: multiple sequential calls in same **transaction**

- ✓ 10a: two consecutive non-view calls – second reflects updated **window**
- ✓ 10b: updateProtectionState **then** non-view – non-view reads **from cache**

E2E-11: atomic **window** expansion + protection **trigger**

- ✓ 11a: spot jumps far above maxPrice – **only** pump triggers, **not** crash

E2E-12: boundary **precision** tests

- ✓ 12a: deviation threshold **strict** inequality – exact boundary does **NOT trigger**
- ✓ 12b: **reset** threshold **strict** inequality – exact **value** reverts, minus 1 succeeds
- ✓ 12c: deadband **strict** inequality – exact **value returns false**, +1 returns true

E2E-13: transient **cache** call-count verification

- ✓ 13a: **cache** hit – update + both views makes only 1 oracle **call**
- ✓ 13b: **cache** miss – views without update make 2 oracle calls
- ✓ 13c: non-view **after update** uses **cache** – update + non-view makes 1 oracle **call**
- ✓ 13d: two consecutive non-view calls – 1 oracle **call** (second reads **cache**)
- ✓ 13e: per-asset **cache** isolation – update A, view B still calls oracle
- ✓ 13f: **cache** hit under active protection – still only 1 oracle **call**
- ✓ 13g: non-whitelisted – **update** fetches+caches, views **read cache = 1** oracle **call**

E2E-14: getBoundedPrices dual-price API

- ✓ 14a: updateProtectionState → getBoundedPricesView **returns both** cached prices
- ✓ 14b: getBoundedPrices (non-view) **returns same as** individual **functions**
- ✓ 14c: getBoundedPricesView matches individual **view functions** (38ms)

E2E-15: keeper updates during active protection

- ✓ 15a: **trigger** → keeper updates min → bounded collateral reflects **new** min
- ✓ 15b: **trigger** → keeper updates max → bounded debt reflects **new** max

E2E-16: **volatile** price extends protection period

- ✓ 16a: continued deviation extends cooldown, blocks early **disable** (44ms)
- ✓ 16b: price normalizes – **no** event, **timestamp** unchanged, protection still active
- ✓ 16c: **full volatile cycle** – repeated spikes extend protection, normalization **in between**, final **disable** (81ms)

ERC4626Oracle unit tests

deployment

- ✓ revert if FRAX address is 0
- ✓ revert if sFRAX address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if address is not valid sFrax address
- ✓ should **get** correct price of sFrax
- ✓ **cache** exchange rate in transient storage

OneJumpOracle unit tests

deployment

- ✓ revert if correlated token address is 0
- ✓ revert if underlying token address is 0
- ✓ revert if resilient oracle address is 0
- ✓ revert if intermediate oracle address is 0
- ✓ should deploy contract


```
getPrice
  ✓ revert if address is not valid LDO address
  ✓ should get correct price of LDO
```

PendleOracle unit tests

```
deployment
  ✓ revert if market address is 0
  ✓ revert if ptOracle address is 0
  ✓ revert if PT token address is 0
  ✓ revert if underlying token address is 0
  ✓ revert if ResilientOracle address is 0
  ✓ revert if TWAP duration is 0
  ✓ revert if invalid TWAP duration

getPrice
  ✓ revert if getPrice argument is not the configured PT token
  ✓ should get correct price for PT_TO_ASSET rate kind
  ✓ should get correct price for PT_TO_SY rate kind
  ✓ should adjust for underlying decimals
```

ReferenceOracle

```
setOracle
  ✓ reverts if the asset is zero address
  ✓ reverts if called by a non-admin
  ✓ emits OracleConfigured event
  ✓ sets the new oracle
  ✓ can unset the oracle

getPrice
  ✓ returns resilient oracle price if there's no oracle configured
  ✓ returns the other oracle price if there's an oracle set for asset

getPriceAssuming
  ✓ returns the assumed price if no oracle is configured
  ✓ returns the assumed price even if the oracle is configured
```

Oracle plugin frame unit tests

```
token config
  add single token config
    ✓ vToken can't be zero & main oracle can't be zero
    ✓ reset token config (50ms)
    ✓ token config added successfully & events check (38ms)

batch add token configs
  ✓ length check
  ✓ token config added successfully & data check (108ms)

change oracle
  set oracle
    ✓ null check (47ms)
    ✓ existence check
    ✓ oracle set successfully & data check (52ms)

get underlying price
  ✓ revert when protocol paused
  ✓ revert price when main oracle is disabled and there is no fallback oracle
  ✓ revert price main oracle returns 0 and there is no fallback oracle
  ✓ revert if price fails checking
  ✓ check price with/without pivot oracle
  ✓ disable pivot oracle
  ✓ enable fallback oracle (44ms)
  ✓ Return fallback price when fallback price is validated successfully with pivot oracle
  ✓ Return main price when fallback price validation failed with pivot oracle
  ✓ Return VAI price
  ✓ Return cached price (40ms)
```

SFraxOracle unit tests

```
deployment
  ✓ revert if FRAX address is 0
  ✓ revert if sFRAX address is 0
  ✓ should deploy contract

getPrice
  ✓ revert if address is not valid sFrax address
  ✓ should get correct price of sFrax
```

SFrxEthOracle unit tests

```
deployment
  ✓ revert if SfrxEthFraxOracle address is 0
```

- ✓ revert if sfrxETH address is 0
- ✓ revert if price different is 0
- ✓ should deploy contract

getPrice

- ✓ revert if address is not valid sfrxETH address
- ✓ revert if price difference is more than allowed
- ✓ should get correct price of sfrxETH

SequencerChainlinkOracle

- ✓ Should revert if sequencer is down
- ✓ Should revert if sequencer is up, but GRACE_PERIOD has not passed
- ✓ Should return price

SlisBNBOracle unit tests

deployment

- ✓ revert if SynclubManager address is 0
- ✓ revert if slisBNB address is 0
- ✓ revert if resilientOracle address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if slisBNB address is wrong
- ✓ should get correct price

StkBNBOracle unit tests

deployment

- ✓ revert if stakePool address is 0
- ✓ revert if stkBNB address is 0
- ✓ revert if resilientOracle address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if ankrBNB address is wrong
- ✓ should get correct price

WBETHOracle unit tests

deployment

- ✓ revert if WBETH address is 0
- ✓ revert if ETH address is 0
- ✓ revert if resilientOracle address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if WBETH address is wrong
- ✓ should get correct price

WeETHOracle unit tests

deployment

- ✓ revert if liquidity pool address is 0
- ✓ revert if weETH address is 0
- ✓ revert if eETH address is 0
- ✓ revert if resilient oracle address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if address is not valid weETH address
- ✓ should get correct price of weETH

WstETHOracleV2 unit tests

deployment

- ✓ revert if wstETH address is 0
- ✓ revert if stETH address is 0
- ✓ revert if ResilientOracle address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if wstETH address is wrong
- ✓ should get correct price

ZkETHOracle

deployment

- ✓ revert if WETH address is 0
- ✓ revert if zkETH address is 0
- ✓ should deploy contract

getPrice

- ✓ revert if address is not valid zkETH address
- ✓ should get correct price of zkETH

```
AsBNBOracle unit tests
  deployment
    ✓ revert if asBNB address is 0
    ✓ revert if slisBNB address is 0
    ✓ should deploy contract
  getPrice
    ✓ revert if asBNB address is wrong
    ✓ should get correct price

CorrelatedTokenOracle
  Initialization
    ✓ validate growth rate
    ✓ validate snapshot
  logic
    ✓ should return the correct price capped by the max allowed price
    ✓ change growth rate (38ms)
    ✓ disable capping
    ✓ update snapshot gap
    ✓ update snapshot
    ✓ zero max allowed exchange rate
```

341 passing (17s)

Code Coverage

npx hardhat coverage

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
contracts/	99.2	89.08	100	98.98	
DeviationBoundedOracle.sol	100	87.96	100	99.46	823
ReferenceOracle.sol	100	87.5	100	100	
ResilientOracle.sol	97.53	91.38	100	97.7	441,470
contracts/interfaces/	100	100	100	100	
FeedRegistryInterface.sol	100	100	100	100	
IAccountant.sol	100	100	100	100	
IAnkrBNB.sol	100	100	100	100	
IAsBNB.sol	100	100	100	100	
IAsBNBMinter.sol	100	100	100	100	
ICappedOracle.sol	100	100	100	100	
IDeviationBoundedOracle.sol	100	100	100	100	
IERC4626.sol	100	100	100	100	
IEtherFiLiquidityPool.sol	100	100	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
IPStakePool.sol	100	100	100	100	
IPendlePtOracle.sol	100	100	100	100	
ISFrax.sol	100	100	100	100	
ISfrxEthFraxOracle.sol	100	100	100	100	
IStETH.sol	100	100	100	100	
IStaderStakeManager.sol	100	100	100	100	
ISynclubStakeManager.sol	100	100	100	100	
IWBETH.sol	100	100	100	100	
IZkETH.sol	100	100	100	100	
OracleInterface.sol	100	100	100	100	
PublicResolverInterface.sol	100	100	100	100	
SIDRegistryInterface.sol	100	100	100	100	
VBep20Interface.sol	100	100	100	100	
contracts/lib/	100	100	100	100	
Transient.sol	100	100	100	100	
contracts/oracles/	85.25	76.47	81.43	83.41	
AnkrBNBOracle.sol	100	100	100	100	
AsBNBOracle.sol	100	100	100	100	
BNBxOracle.sol	100	100	100	100	
BinanceOracle.sol	87.8	71.43	100	95.74	130,131
BoundValidator.sol	96.15	83.33	100	95.24	112
ChainlinkOracle.sol	100	92.31	100	100	
ERC4626Oracle.sol	100	100	100	100	
EtherfiAccountantOracle.sol	0	100	0	0	43,44,52
OneJumpOracle.sol	100	100	100	100	
PendleOracle.sol	100	83.33	100	100	
SFraxOracle.sol	100	100	100	100	
SFrxEthOracle.sol	100	75	100	100	

File	% Stmts	% Branch	% Funcs	% Lines	Uncovered Lines
SequencerChainlinkOracle.sol	100	87.5	100	100	
SlisBNBOracle.sol	100	100	100	100	
StableUsdtPriceFeed.sol	0	0	0	0	... 49,53,62,63
StkBNBOracle.sol	100	50	100	83.33	61
WBETHOracle.sol	100	100	100	100	
WeETHAccountantOracle.sol	0	100	0	0	42,43,51
WeETHOracle.sol	100	100	100	100	
WstETHOracle.sol	0	0	0	0	... 64,67,70,73
WstETHOracleV2.sol	100	100	100	100	
ZkETHOracle.sol	100	100	100	100	
contracts/oracles/common/	100	93.33	100	98.41	
CorrelatedTokenOracle.sol	100	93.33	100	98.41	270
All files	93.96	85.29	91.5	93.09	

Changelog

- 2026-04-27 - Initial report
- 2026-05-05 - Final report

About Quantstamp

Quantstamp is a global leader in blockchain security. Founded in 2017, Quantstamp's mission is to securely onboard the next billion users to Web3 through its best-in-class Web3 security products and services.

Quantstamp's team consists of cybersecurity experts hailing from globally recognized organizations including Microsoft, AWS, BMW, Meta, and the Ethereum Foundation. Quantstamp engineers hold PhDs or advanced computer science degrees, with decades of combined experience in formal verification, static analysis, blockchain audits, penetration testing, and original leading-edge research.

To date, Quantstamp has performed more than 1300 audits and secured over \$500 billion in digital asset risk from hackers. Quantstamp has worked with a diverse range of customers, including startups, category leaders and financial institutions. Brands that Quantstamp has worked with include Ethereum 2.0, Binance, Visa, PayPal, Solana, Canton, Polymarket, PancakeSwap, Virtuals Protocol, Wayfinder, Lido, TrustWallet, Alchemy, World Economic Forum.

Quantstamp's collaborations and partnerships showcase our commitment to world-class research, development and security. We're honored to work with some of the top names in the industry and proud to secure the future of web3.

Notable Collaborations & Customers:

- Blockchains: Ethereum 2.0, Solana, Polygon, TON, Cardano, Binance Smart Chain, Avalanche, Arbitrum, Flow
- DeFi: Lido, Ethena, Compound, Curve, Venus, PancakeSwap, Polymarket
- Infra/Staking: TrustWallet, Alchemy, Liquid Collective, Kiln, Galxe, API3, ssv.network, Luganodes
- Immersive: Virtuals Protocol, Wayfinder, OpenSea, Square Enix, Parallel, Camp, Decentraland
- Institutional: Visa, Circle, Revolut, Anthropic, OpenAI, Canton, Republic, Arkham, Sequoia

Timeliness of content

The content contained in the report is current as of the date appearing on the report and is subject to change without notice, unless indicated otherwise by Quantstamp; however, Quantstamp does not guarantee or warrant the accuracy, timeliness, or completeness of any report you

access using the internet or other means, and assumes no obligation to update any information following publication or other making available of the report to you by Quantstamp.

Notice of confidentiality

This report, including the content, data, and underlying methodologies, are subject to the confidentiality and feedback provisions in your agreement with Quantstamp. These materials are not to be disclosed, extracted, copied, or distributed except to the extent expressly authorized by Quantstamp.

Links to other websites

You may, through hypertext or other computer links, gain access to web sites operated by persons other than Quantstamp. Such hyperlinks are provided for your reference and convenience only, and are the exclusive responsibility of such web sites' owners. You agree that Quantstamp are not responsible for the content or operation of such web sites, and that Quantstamp shall have no liability to you or any other person or entity for the use of third-party web sites. Except as described below, a hyperlink from this web site to another web site does not imply or mean that Quantstamp endorses the content on that web site or the operator or operations of that site. You are solely responsible for determining the extent to which you may use any content at any other web sites to which you link from the report. Quantstamp assumes no responsibility for the use of third-party software on any website and shall have no liability whatsoever to any person or entity for the accuracy or completeness of any output generated by such software.

Disclaimer

The review and this report are provided on an as-is, where-is, and as-available basis. To the fullest extent permitted by law, Quantstamp disclaims all warranties, expressed implied, in connection with this report, its content, and the related services and products and your use thereof, including, without limitation, the implied warranties of merchantability, fitness for a particular purpose, and non-infringement. You agree that access and/or use of the report and other results of the review, including but not limited to any associated services, products, protocols, platforms, content, and materials, will be at your sole risk. FOR AVOIDANCE OF DOUBT, THE REPORT, ITS CONTENT, ACCESS, AND/OR USAGE THEREOF, INCLUDING ANY ASSOCIATED SERVICES OR MATERIALS, SHALL NOT BE CONSIDERED OR RELIED UPON AS ANY FORM OF FINANCIAL, INVESTMENT, TAX, LEGAL, REGULATORY, OR OTHER ADVICE. This report is based on the scope of materials and documentation provided for a limited review at the time provided. You acknowledge that Blockchain technology remains under development and is subject to unknown risks and flaws and, as such, the report may not be complete or inclusive of all vulnerabilities. The review is limited to the materials identified in the report and does not extend to the compiler layer, or any other areas beyond the programming language, or programming aspects that could present security risks. The report does not indicate the endorsement by Quantstamp of any particular project or team, nor guarantee its security, and may not be represented as such. No third party is entitled to rely on the report in any way, including for the purpose of making any decisions to buy or sell a product, service or any other asset. Quantstamp does not warrant, endorse, guarantee, or assume responsibility for any product or service advertised or offered by a third party, or any open source or third-party software, code, libraries, materials, or information to, called by, referenced by or accessible through the report, its content, or any related services and products, any hyperlinked websites, or any other websites or mobile applications, and we will not be a party to or in any way be responsible for monitoring any transaction between you and any third party. As with the purchase or use of a product or service through any medium or in any environment, you should use your best judgment and exercise caution where appropriate.

